# SPEECHMASTER: COMPLEMENTARITY-AWARE ROUTING FOR SELF-SUPERVISED ASR WITH DISCRETE UNIT BUDGETING

*Zihang Zhou*[⋆†] *Kewei Liu*[⋆†] *Zihao Long*[⋆†]

[⋆]Shanghai Jiao Tong University [†]Equal contribution

## ABSTRACT

Self-supervised speech encoders are strong ASR front ends, but deploying the largest recognizer on every utterance wastes computation and obscures the accuracy/representation-cost tradeoff. We present SpeechMaster, a budget-aware ASR framework that composes heterogeneous self-supervised representations. A fast Wav2Vec2 CTC branch decodes every utterance, a complementarity-aware router predicts the edit-count gain of rerouting to HuBERT, and a HuBERT unit auditor measures token rate and bitrate under discrete codebooks. The router is trained only from cached dev-clean branch outputs and uses fast-branch features at inference. On 1024 LibriSpeech dev-clean utterances, routing only 25% of utterances reduces WER from 2.94% to 2.18%, while full HuBERT decoding reaches 1.82%. On 1024 test-clean utterances, the frozen router improves WER from 3.61% to 2.78% at a 25% routing budget and to 2.40% at a 50% budget. An oracle route gives 1.45% dev WER and 1.92% test WER, showing complementary SSL-ASR errors.

***Index Terms***— ASR, self-supervised speech, routing, HuBERT, Wav2Vec2

## 1. INTRODUCTION

Low-resource ASR benefits from self-supervised learning (SSL) because encoders such as Wav2Vec2, HuBERT, and WavLM learn phonetic and lexical structure from unlabeled audio. A common downstream design selects one SSL checkpoint, attaches or reuses a CTC decoder, and reports WER. This single-model view misses a practical question: many utterances are easy for a fast recognizer, while a smaller subset needs a stronger model. A useful low-resource system should therefore expose an accuracy/compute knob instead of committing to one encoder.

SpeechMaster treats SSL-ASR as a routed composition. It always runs a fast Wav2Vec2 branch, predicts the utterance-level gain of replacing that hypothesis with HuBERT, and spends the HuBERT large branch only where the predicted gain is largest under a user-selected budget. In parallel, a discrete-unit auditor clusters HuBERT hidden states to quantify the bitrate needed by unit-based downstream variants. The contributions are: (1) a complementarity-aware routing target that learns from cached fast/strong branch disagreement, (2) a lightweight gain predictor using only first-pass CTC, duration, and transcript-rate features at inference, (3) a discrete-unit bitrate auditor over HuBERT layers and codebook sizes, and (4) reproducible dev/test experiments with WER, CER, RTF, token rate, bitrate, confidence-router baselines, and CAR feature ablations.

## 2. RELATED WORK

Wav2Vec2 learns speech representations with a contrastive objective over masked latent frames, making it effective when downstream labels are scarce. HuBERT instead predicts masked cluster assignments and often produces strong phonetic-content representations. WavLM extends masked speech pretraining with denoising and full-stack speech processing objectives. These SSL encoders are usually evaluated as independent ASR backbones. SpeechMaster is orthogonal: it does not propose another encoder, but composes existing SSL recognizers under a budget.

Confidence-based cascades have been used to avoid expensive inference on easy examples. Standard confidence measures, however, are often calibrated for one model and do not ask whether another recognizer will actually fix the error. SpeechMaster keeps CTC confidence as a reproducible baseline, but its main router learns branch complementarity: the target is the edit-count reduction obtained by replacing the fast hypothesis with the strong one. Discrete speech units are also widely used in speech generation and unit-based recognition. Our unit auditor is not a new tokenizer; it reports the information rate induced by layer and codebook choices, making routed ASR comparable with unit-based SSL systems. Open-source SSL speech toolkits make these systems easier to reproduce; our code follows the same reproducibility spirit but focuses on the routing and representation-budget layer.

## 3. SPEECHMASTER

### 3.1. System Overview

Fig. 1 shows the complete SpeechMaster pipeline. The first pass is a fast SSL-ASR decoder that produces both a transcript and a confidence trace. A complementarity-aware router converts fast-branch features into an estimated HuBERT edit-gain and spends the strong branch only on the top-ranked fraction requested by the compute budget. The final transcript is therefore a mixture of fast and strong hypotheses. A separate unit auditor reads HuBERT hidden states and reports the bitrate of discrete unit streams; it is an analysis branch and does not change ASR output.

### 3.2. Two-Branch SSL-ASR

Given waveform $x$, a pretrained SSL encoder $f_\theta$ produces contextual hidden states $H = f_\theta(x)$. A CTC projection [6] maps each frame to token posteriors:

$$p_\theta(y_t \mid x) = \mathrm{softmax}(WH_t + b). \tag{1}$$

SpeechMaster uses Wav2Vec2 base as the fast branch $F$ and HuBERT large as the strong branch $S$. Both are public SSL-ASR checkpoints; no ASR encoder or CTC head is fine-tuned. The only trained component is the small CAR router described below.

### 3.3. Complementarity-Aware Router

Let $q_t$ denote the fast-branch posterior and let $\mathcal{T}$ be non-blank argmax frames. A label-free confidence baseline scores the fast hypothesis by posterior peakiness minus entropy:

$$c(x) = \frac{1}{|\mathcal{T}|}\sum_{t\in\mathcal{T}} \max_k q_t(k) - 0.03\frac{1}{|\mathcal{T}|}\sum_{t\in\mathcal{T}} H(q_t). \tag{2}$$

SpeechMaster-CAR goes beyond this heuristic by learning whether HuBERT is expected to reduce the fast-branch edit count. On cached development utterances with reference $y$, the training target is

$$g^\star(x) = e(F(x), y) - e(S(x), y), \tag{3}$$

where $e(\cdot,\cdot)$ is word edit count. Positive $g^\star$ means the strong branch fixes more errors than it introduces. The router extracts fast-only features $\phi(x)$: $c(x)$, entropy, posterior peak, duration, log-duration, first-pass word/character counts, word/character rates, and risk-duration interactions. A standardized ridge regressor predicts $\hat{g}(x) = w^\top z(\phi(x))$. At budget $B$, utterances with the largest predicted gain are decoded by $S$; all others keep the fast hypothesis:

$$\hat{y}(x) = \begin{cases} S(x), & x \in \mathrm{TopB}(\hat{g}), \\ F(x), & \text{otherwise.} \end{cases} \tag{4}$$

The same saved branch outputs are reused to ablate peak-only, entropy-only, duration, random, original confidence routing, CAR feature groups, and oracle routers.

### 3.4. Discrete Unit Auditor

To connect ASR accuracy with representation compression, SpeechMaster extracts HuBERT hidden states from several layers and trains MiniBatchKMeans codebooks [7]. For codebook size $K$ and token rate $r_{\mathrm{tok}}$, the nominal bitrate is

$$\mathrm{bitrate} = r_{\mathrm{tok}} \log_2 K. \tag{5}$$

We also collapse consecutive duplicate units, giving a compact token stream useful for text-to-unit or unit-to-ASR analyses.

## 4. EXPERIMENTAL SETUP

Experiments use LibriSpeech clean splits at 16 kHz. The main development suite uses 1024 dev-clean utterances; the final validation suite uses 1024 test-clean utterances. Unit analysis uses 512 dev-clean utterances. The fast branch is Wav2Vec2-Base-960h; the strong branch is HuBERT-Large-960h; a stronger reference model is Wav2Vec2-Large-LV60. HuBERT unit analysis uses HuBERT-Base. Exact checkpoint identifiers are listed in the released scripts. Metrics are WER, CER, and real-time factor (RTF) for ASR, plus token rate, deduplicated token rate, bitrate, and codebook size for discrete units.

All decoding uses greedy CTC to isolate representation and routing effects from external language-model gains. We save utterance-level hypotheses, durations, branch confidences, and decode times as JSONL files. Every budget point is then deterministic after branch decoding: a new router can be evaluated by reordering the same utterances rather than rerunning ASR. CAR is trained on the cached dev-clean branch outputs and frozen for test-clean. The heavy suite therefore separates neural inference from cheap routing and feature ablations.

## 5. RESULTS

### 5.1. Main Dev-Clean Results

Table 1 is the main SpeechMaster development result. At a 25% routing budget, CAR drops WER from 2.94% to 2.18%, a relative reduction of 25.9%, while using substantially less strong-branch decoding than 100% HuBERT. At 50%, WER reaches 2.01%. The oracle route is lower than both branches on dev-clean, showing that fast and strong SSL recognizers make non-identical errors.

Table 2 reports single-model and layer ablations. The final Wav2Vec2 CTC layer is stronger than last-layer averaging or mid/final blending, indicating that the pretrained CTC head is calibrated to the final state. The large LV60 Wav2Vec2 model is the best single model on this dev slice, but the fast+HuBERT oracle still improves over it.

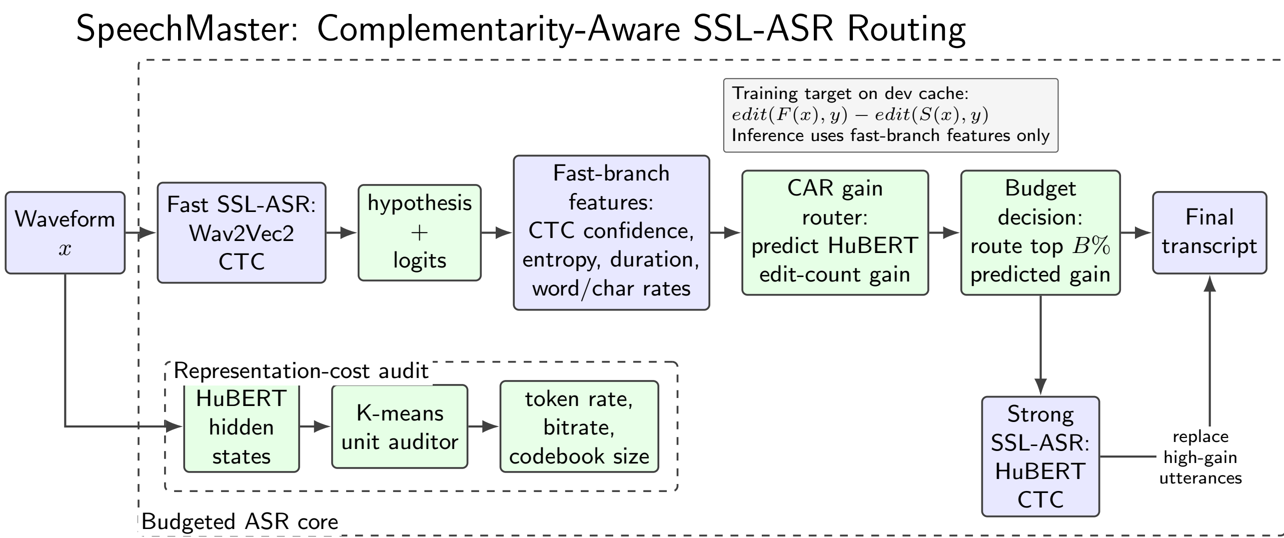


**Fig. 1**. Overview of SpeechMaster. CAR predicts where HuBERT will improve the fast SSL-ASR transcript, while the unit auditor quantifies discrete SSL representation cost.

**Table 1**. SpeechMaster-CAR routing on 1024 LibriSpeech dev-clean utterances.

| Route | Routed utt. | WER | CER | RTF |
|---|---|---|---|---|
| 0% | 0 | 2.943 | 0.990 | 0.001 |
| 10% | 102 | 2.516 | 0.817 | 0.001 |
| 25% | 256 | 2.182 | 0.692 | 0.002 |
| 50% | 512 | 2.010 | 0.636 | 0.002 |
| 75% | 768 | 1.887 | 0.603 | 0.002 |
| 100% | 1024 | 1.823 | 0.575 | 0.003 |
| Oracle | - | 1.450 | 0.520 | - |

**Table 2**. Single-model ASR and layer ablations on dev-clean.

| System | Rep. | N | WER | CER | RTF |
|---|---|---|---|---|---|
| wav2vec2-base-960h | final | 1024 | 2.943 | 0.990 | 0.001 |
| wav2vec2-base-960h | last4 | 1024 | 3.327 | 1.111 | 0.002 |
| wav2vec2-base-960h | midlast | 1024 | 3.705 | 1.140 | 0.001 |
| hubert-large-ls960-ft | final | 1024 | 1.823 | 0.575 | 0.003 |
| wav2vec2-large-960h-lv60-self | final | 1024 | 1.636 | 0.532 | 0.003 |

## 5.2. Test-Clean Verification

Table 3 verifies the same framework on test-clean. The absolute WERs are higher than dev-clean, but the monotonic routing behavior remains: 25% routing improves WER from 3.61% to 2.78%, and 50% routing reaches 2.40%. The oracle route reaches 1.92%, confirming remaining headroom for better gain prediction.

**Table 3**. SpeechMaster-CAR routing on 1024 LibriSpeech test-clean utterances.

| Route | Routed utt. | WER | CER | RTF |
|---|---|---|---|---|
| 0% | 0 | 3.614 | 1.186 | 0.001 |
| 10% | 102 | 3.205 | 1.005 | 0.001 |
| 25% | 256 | 2.777 | 0.846 | 0.002 |
| 50% | 512 | 2.405 | 0.731 | 0.002 |
| 75% | 768 | 2.315 | 0.694 | 0.002 |
| 100% | 1024 | 2.231 | 0.667 | 0.002 |
| Oracle | - | 1.920 | 0.658 | - |

## 5.3. Router and Unit Ablations

Routing-score ablations show that risk-aware ordering is consistently better than random routing. On dev-clean at 25%, random routing gives 2.66% WER, duration gives 2.54%, entropy gives 2.31%, the original peak-entropy router gives 2.40%, and CAR gives 2.18%; the oracle gives 1.45%. On test-clean at 50%, random gives 2.93%, duration gives 2.62%, peak-only gives 2.59%, peak-entropy gives 2.60%, and CAR gives 2.40%. Thus CAR improves the WER/RTF frontier without changing either SSL recognizer.

The complementarity counts support the same conclusion. On dev-clean, the fast branch is better for 67 utterances, the strong branch is better for 222, and 735 are tied at utterance-level WER. On test-clean, the corresponding counts are 53, 252, and 719. A router that always trusts the strong model cannot exploit the first group; a router that only uses the fast model misses the second group. SpeechMaster's bud-

**Table 4**. CAR feature ablation on 1024 test-clean utterances.

| Router features | 25% WER | 50% WER |
|---|---|---|
| CTC only | 3.03 | 2.64 |
| Duration only | 2.95 | 2.63 |
| CTC + duration | 2.78 | 2.44 |
| Full CAR | 2.78 | 2.40 |

**Table 5**. Selected HuBERT unit statistics on 512 dev-clean utterances.

| Layer | $K$ | Tok/s | Dedup tok/s | Dedup bit/s |
|---|---|---|---|---|
| $-8$ | 100 | 49.90 | 28.08 | 186.58 |
| $-8$ | 1000 | 49.90 | 35.17 | 350.46 |
| $-4$ | 100 | 49.90 | 27.14 | 180.34 |
| $-4$ | 1000 | 49.90 | 29.74 | 296.36 |
| $-1$ | 100 | 49.90 | 26.67 | 177.19 |
| $-1$ | 1000 | 49.90 | 34.50 | 343.85 |

geted routing is a practical middle point between these two extremes.

The unit auditor exposes a compression tradeoff. Raw HuBERT assignments appear at about 49.9 tokens/s for all tested layers. Deduplication roughly halves the rate: for example, layer $-4$ with $K = 100$ yields 27.14 deduplicated tokens/s and 180.34 bit/s, while $K = 1000$ raises this to 29.74 tokens/s and 296.36 bit/s. Layer $-8$ keeps more rapidly changing units than layer $-4$ at the same codebook size, suggesting a higher acoustic-detail bitrate. A companion WavLM frozen-probe implementation further trains a 2.84M-parameter BiLSTM-CTC head with 10h labels: continuous layer-10 features reach 12.86% test-clean WER, while k-means units at $K = 500$ reach 23.90% WER at 448 bit/s. Thus discrete units are compact and usable, but continuous states preserve more ASR detail.

# 6. DISCUSSION

## 6.1. Why SpeechMaster is More Than Model Reuse

The assignment encourages pretrained SSL models, but the system should not stop at reproducing a public checkpoint. SpeechMaster adds a decision layer around SSL-ASR: it learns which representation should be trusted for each utterance. This changes the downstream task from "choose one recognizer" to "allocate recognition compute under a budget." If HuBERT dominated Wav2Vec2 on every utterance, oracle routing would equal full HuBERT. Instead, both branches win on different utterances. CAR exploits this structure by predicting edit reduction from first-pass features, improving over confidence-only routing while leaving a clear oracle gap for richer gain models.

## 6.2. Deployment Interpretation

SpeechMaster is useful when recognition runs over large audio collections. At $B = 0$, it behaves like fast Wav2Vec2; at $B = 100$, like HuBERT large. Intermediate budgets trade a small RTF increase for a smooth WER reduction, unlike ensembling which evaluates every branch on every utterance. Cached branch outputs also make offline development cheap: CAR training, confidence baselines, and feature ablations can be tested without rerunning neural inference. The unit auditor adds the representation-compression axis, e.g., layer $-4$, $K = 100$ gives about 180 bit/s after deduplication while $K = 1000$ gives more acoustic detail.

## 6.3. Limitations

The main limitation is router generalization. CAR is small and transparent, but it is trained on clean dev utterances and does not model speaker condition, accent, or acoustic domain shift. Noisy or accented speech may increase both branch disagreement and routing value. The unit auditor also reports compression statistics rather than directly training a unit-based recognizer; coupling the selected codebooks to a decoder is the natural next step.

# 7. CONCLUSION

SpeechMaster combines fast and strong self-supervised ASR models through complementarity-aware gain routing and audits the bitrate of HuBERT discrete units. Across dev-clean and test-clean, routing a limited fraction of utterances gives a smooth WER/RTF tradeoff, while CAR improves over confidence-only routing. The results support a system-level view of low-resource SSL-ASR in which encoder choice, routing budget, and representation bitrate are optimized jointly.

# 8. REFERENCES


[1] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An ASR corpus based on public domain audio books," in *Proc. ICASSP*, 2015.

[2] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *NeurIPS*, 2020.

[3] W.-N. Hsu et al., "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM TASLP*, 2021.

[4] S. Chen et al., "WavLM: Large-scale self-supervised pre-training for full stack speech processing," *IEEE JSTSP*, 2022.

[5] Jitsi, "JiWER: Word error rate and character error rate for automatic speech recognition," `https://jitsi.github.io/jiwer/`.

[6] A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. ICML*, 2006.

[7] D. Sculley, "Web-scale k-means clustering," in *Proc. WWW*, 2010.